

UPSKILL CAMPUS

in association with UniConverge Technologies (UCT)

INTERNSHIP PROJECT REPORT

on

“Secure Password Manager”

A Desktop Application for Encrypted Credential Management

Submitted by:	Himanshu Rautela
Program:	Upskill Campus – UCT Internship Program
Project Domain:	Python Desktop Application Development
Project Duration:	4 Weeks
Technology Stack:	Python, Tkinter, SQLite, Cryptography (Fernet)
Submission Date:	July 2026

Executive Summary

This report presents the complete design, development, and evaluation of the Secure Password Manager, a desktop application built during a four-week internship conducted under the Upskill Campus program in association with UniConverge Technologies (UCT). The project was undertaken to address a widely recognised problem in everyday digital hygiene: the insecure storage and management of account credentials by end users, who frequently rely on unencrypted notes, browser-based storage, or memory-dependent habits that are vulnerable to compromise.

The Secure Password Manager was designed as a fully offline, self-contained desktop application using Python as the core programming language, Tkinter for the graphical user interface, SQLite as the embedded database engine, and the Fernet symmetric encryption scheme from the cryptography library to protect stored credentials at rest. The application enforces master-password based authentication, encrypts every stored password before it is written to disk, and provides a complete set of credential-management operations including creation, retrieval, modification, deletion, and search.

Beyond core CRUD functionality, the application includes a built-in password generator capable of producing cryptographically strong random passwords, a password-strength evaluation mechanism, a dashboard presenting summary statistics of stored credentials, and a clipboard-copy utility that allows users to transfer a decrypted password to the system clipboard without displaying it directly on screen. The user interface was implemented with a modern dark theme intended to reduce eye strain and align with the visual conventions expected of contemporary security tooling.

This document describes the problem context, the shortcomings of existing password-management approaches, the architecture and internal design of the proposed solution, the database schema, the encryption and authentication workflows, the graphical interface, and the results of functional, performance, and security testing carried out on the completed application. The report closes with a discussion of the advantages and limitations of the current implementation, a roadmap of future enhancements, and a reflection on the technical and professional learnings gained over the course of the internship.

Table of Contents

Executive Summary.....	2
Table of Contents.....	2
Preface.....	4
1. Introduction.....	4
2. About UniConverge Technologies (UCT).....	5
3. About Upskill Campus.....	5

4. Objectives.....	6
5. Problem Statement.....	6
6. Existing Solutions.....	7
6.1 Manual / Physical Methods.....	7
6.2 Browser-Integrated Password Managers.....	7
6.3 Cloud-Based Commercial Password Managers.....	7
6.4 Plain-Text Digital Files.....	8
7. Limitations of Existing Solutions.....	8
8. Proposed Solution.....	8
8.1 Advantages of the Proposed Solution.....	9
9. System Architecture.....	9
9.1 High-Level Architecture Diagram.....	9
9.2 Low-Level Architecture Diagram.....	10
9.3 Data Flow Diagram.....	11
10. Database Design.....	11
10.1 SQLite Schema Explanation.....	11
10.2 Database Design Considerations.....	12
11. Application Workflow.....	12
12. Module Description.....	13
12.1 Module Relationship Diagram.....	14
13. Encryption Workflow.....	15
13.1 Key Generation and Storage.....	15
13.2 Encryption Workflow Diagram.....	15
13.3 Decryption Workflow.....	16
14. Authentication Workflow.....	16
15. GUI Explanation.....	17
16. Screenshots.....	17
17. Testing.....	23
17.1 Functional Testing.....	24
17.2 Test Cases.....	24
17.3 Performance Results.....	25
17.4 Security Testing.....	25
18. Advantages.....	26
19. Limitations.....	26
20. Future Scope.....	26

21. Learnings.....	27
22. Conclusion.....	27
23. References.....	28
24. Glossary.....	28
25. Appendix.....	29
25.1 Code Submission.....	29
25.2 Report Submission.....	30
25.3 Technology Stack Summary.....	30

Preface

This report is submitted in fulfilment of the requirements of the internship program jointly conducted by Upskill Campus and UniConverge Technologies (UCT). The internship provided an opportunity to move beyond classroom instruction and apply core programming, software engineering, and security concepts to the construction of a complete, working desktop application from initial requirements through to a tested, documented deliverable.

As a first-year student pursuing a B.Tech in Computer Science Engineering with a specialisation in Artificial Intelligence and Machine Learning, this internship represented an early and valuable exposure to software engineering practice outside the AI/ML domain — specifically, application security, desktop GUI development, embedded database design, and defensive programming. The skills exercised in this project, including secure coding practices, encryption fundamentals, and structured software documentation, form a foundation that is directly transferable to future work across software domains, including AI/ML systems that must themselves handle sensitive data responsibly.

I would like to acknowledge the guidance and structured curriculum provided by Upskill Campus and the mentorship and industry context provided by UniConverge Technologies, both of which made this internship a productive and instructive experience.

1. Introduction

In the contemporary digital environment, an average internet user maintains active accounts across dozens of platforms — email services, banking portals, social media networks, academic systems, and workplace tools — each of which requires a distinct, sufficiently complex password to remain secure against credential-stuffing and brute-force attacks. The cognitive burden of remembering dozens of unique, high-entropy passwords leads the majority of users toward insecure coping mechanisms: reusing the same password across multiple services, writing passwords down in physical notebooks, storing them

in unencrypted text files or spreadsheets, or relying on browser-integrated password storage that is often only weakly protected.

Each of these coping mechanisms introduces a distinct security vulnerability. Password reuse means that a single data breach on one platform can cascade into account compromise across every other service where the same credential was used. Plain-text storage, whether on paper or in digital files, offers no protection at all against anyone with physical or file-system access to the storage medium. Even browser-based password managers, while more convenient, frequently store credentials with weak or absent encryption keys tied to the operating system login, offering limited protection if the device itself is compromised.

The Secure Password Manager project was conceived directly in response to this problem space. The objective was to design and implement a lightweight, fully offline desktop application that allows a single user to store, retrieve, and manage an arbitrary number of account credentials behind a single master password, with every stored password encrypted at rest using industry-standard symmetric encryption. Unlike cloud-based password managers, the application stores all data locally in an SQLite database file, eliminating any dependency on third-party servers, network connectivity, or external trust relationships, and giving the user complete custody of their credential data.

This report documents the complete lifecycle of the project: the problem analysis that motivated the design, the evaluation of existing solutions and their limitations, the proposed architecture and its component modules, the database and encryption design, the graphical user interface, and the functional, performance, and security testing performed to validate the finished application.

2. About UniConverge Technologies (UCT)

UniConverge Technologies Pvt. Ltd. (UCT) is a technology solutions and IoT-focused organisation engaged in delivering embedded systems, edge computing, and enterprise digital transformation services. UCT collaborates with academic institutions and skill-development platforms such as Upskill Campus to design structured, industry-aligned internship programs that expose students to practical software and systems engineering problems drawn from real deployment contexts.

As an industry partner in this internship program, UCT contributes domain expertise, project frameworks, and technical mentorship, with the objective of bridging the gap between academic curricula and the practical engineering discipline expected in professional software development roles. UCT's involvement in student internship programs reflects a broader industry commitment to early-career talent development in embedded systems, IoT, and applied software engineering.

3. About Upskill Campus

Upskill Campus is an ed-tech and skilling platform that partners with industry organisations to deliver structured, project-based internship and certification programs to engineering students. The platform's internship model is built around the completion of a defined technical project over a fixed duration, supported by curated learning material, milestone-based progress tracking, and a formal reporting and evaluation process culminating in a certificate of completion.

The Upskill Campus program model emphasises applied, outcome-based learning: rather than purely theoretical instruction, interns are expected to design, build, test, and document a functioning software artefact, closely mirroring the expectations of an entry-level industry software engineering role. This internship report, and the accompanying Secure Password Manager application, constitute the primary deliverable of that program for this internship cycle.

4. Objectives

The Secure Password Manager project was guided by the following core objectives:

- To design and implement a secure, offline, desktop-based password management application that eliminates dependency on cloud infrastructure or third-party servers.
- To protect all stored user credentials using strong, industry-recognised symmetric encryption (Fernet, built on AES-128 in CBC mode with HMAC authentication) so that credential data is unreadable outside the application, even if the underlying database file is accessed directly.
- To provide a clean, intuitive, and modern graphical interface using Tkinter that allows non-technical users to manage credentials without requiring command-line interaction.
- To replace insecure, ad-hoc credential-storage habits (physical notebooks, unencrypted text files, browser autofill) with a purpose-built, security-first alternative.
- To implement complete CRUD (Create, Read, Update, Delete) functionality for credential records, along with search, filtering, and dashboard-level visibility into stored data.
- To incorporate a cryptographically sound password generator and a password-strength evaluation mechanism to actively encourage the use of strong, unique passwords for every account.
- To validate the application through structured functional, performance, and security testing, and to document the complete engineering process in a professional, industry-standard report.

5. Problem Statement

The majority of internet users manage between 20 and 100 distinct online accounts, each nominally requiring a strong, unique password. In practice, human memory limitations make it impractical to reliably recall this many high-entropy secrets, and this gap between security best practice and human cognitive capacity is the root cause of widespread credential insecurity. The problem can be stated formally as

follows: users require a mechanism to store an arbitrary number of account credentials such that (a) the credentials are protected against unauthorised access even if the storage medium is compromised, (b) the credentials remain conveniently and quickly retrievable by the legitimate user, and (c) the mechanism does not introduce new points of failure such as dependency on a third-party cloud provider, a persistent internet connection, or a proprietary, closed storage format.

Existing informal practices — physical notebooks, sticky notes, unencrypted spreadsheets, and password reuse — satisfy convenience at the direct expense of security. Existing cloud-based commercial solutions typically satisfy security and convenience but at the cost of local data custody, recurring subscription cost, and reliance on the continued operation and trustworthiness of a third-party vendor. The Secure Password Manager project addresses the specific gap of a lightweight, free, fully local, and genuinely encrypted password storage tool suitable for an individual user's personal device.

6. Existing Solutions

Several categories of existing credential-management approaches were surveyed prior to designing the proposed solution:

6.1 Manual / Physical Methods

Many users record passwords in physical notebooks, diaries, or sticky notes affixed to their workstation. This approach requires no technical infrastructure but offers no protection against physical theft, loss, or casual observation, and does not scale beyond a small number of accounts.

6.2 Browser-Integrated Password Managers

Modern web browsers such as Chrome, Firefox, and Edge include built-in credential storage and autofill functionality. While convenient, browser-stored passwords are typically encrypted using a key derived from the operating system user profile, meaning that anyone with access to a logged-in user session, or in some configurations the raw profile files, can potentially extract stored passwords in plain text.

6.3 Cloud-Based Commercial Password Managers

Dedicated commercial products (such as well-known SaaS password vaults) offer strong end-to-end encryption, cross-device synchronisation, and browser integration. These solutions are generally well engineered from a security standpoint but require a recurring subscription, dependency on the vendor's continued operation, transmission of encrypted vault data to third-party servers, and creation of an account with an external provider — all of which represent trust and availability trade-offs that some users are unwilling to accept.

6.4 Plain-Text Digital Files

Storing credentials in an unencrypted spreadsheet or text document is a common but highly insecure practice. Any malware, unauthorised device access, or accidental cloud-sync exposure immediately compromises every credential stored in the file, with no cryptographic barrier whatsoever.

7. Limitations of Existing Solutions

Existing Approach	Key Limitation
Physical notebook / sticky notes	No protection against physical theft or loss; does not scale; no backup mechanism.
Browser-integrated storage	Weak encryption tied to OS session; vulnerable to local extraction tools; limited organisation features.
Cloud-based commercial managers	Recurring cost; dependency on third-party servers; vault data leaves the local device; account-based trust model.
Unencrypted digital files	Zero cryptographic protection; single point of catastrophic failure if the file is accessed.
Password reuse across accounts	A single breach compromises every account sharing the reused credential.

Across all surveyed approaches, no informal or free option provided genuine, local, encryption-backed credential storage without requiring a subscription or external service dependency. This gap defined the specific design target for the Secure Password Manager: local-first, encrypted, free, and self-contained.

8. Proposed Solution

The Secure Password Manager is proposed as a standalone Python desktop application that runs entirely on the user's local machine with no network dependency. All credential data is stored in a local SQLite database file, with every password field encrypted using the Fernet symmetric encryption scheme before being written to disk. Access to the application and its stored credentials is gated behind a single master password, which the user establishes on first run and must supply on every subsequent launch.

The proposed solution comprises the following functional pillars:

- Master Password Authentication — a single, memorable secret that unlocks access to the encrypted credential vault.
- Encrypted Storage — every password is encrypted with Fernet (AES-128-CBC + HMAC-SHA256) prior to persistence in SQLite, and decrypted only in-memory at the point of display or copy.
- Full CRUD Operations — add, view, edit, delete, and search credential records through the GUI.

- Password Generator — on-demand generation of cryptographically strong random passwords with configurable length and character composition.
- Password Strength Indicator — real-time feedback on the strength of a password being entered or generated.
- Dashboard — a summary view presenting the total number of stored credentials, an aggregate password-strength score, and the timestamp of the most recent update.
- Clipboard Integration — one-click copy of a decrypted password to the system clipboard, avoiding unnecessary on-screen exposure.
- Modern Dark-Themed UI — a visually clean Tkinter interface designed for everyday practical use.

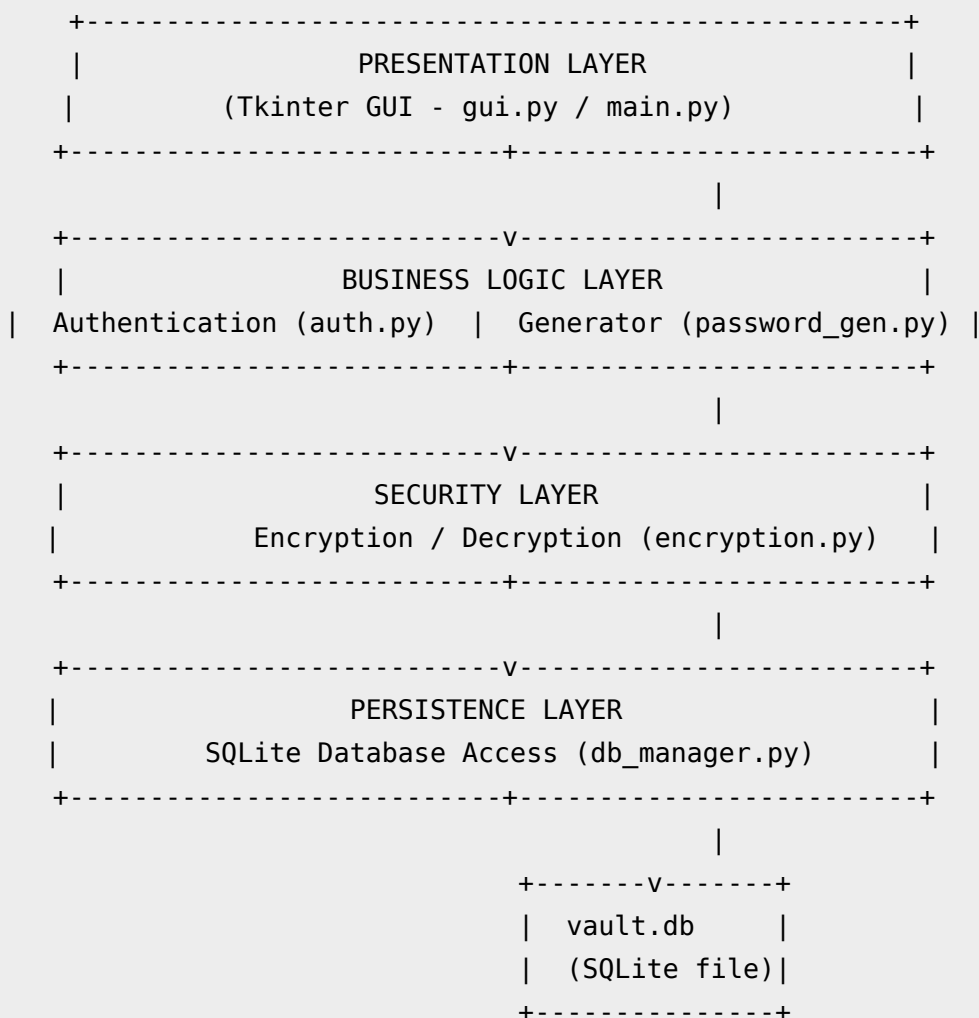
8.1 Advantages of the Proposed Solution

Advantage	Description
Complete data ownership	All data remains on the user's own device; nothing is transmitted to external servers.
No recurring cost	The application is free to use indefinitely, with no subscription model.
Strong local encryption	Fernet-based encryption ensures stored passwords are unreadable without the master key, even with direct file access.
Offline operability	The application functions with no internet connection, removing network-based attack surface.
Lightweight footprint	Built on Python, Tkinter, and SQLite, the application has minimal system resource requirements.
Extensibility	The modular architecture allows future addition of features such as biometric unlock or cloud backup without redesigning the core system.

9. System Architecture

The Secure Password Manager follows a layered desktop-application architecture that cleanly separates the presentation layer (Tkinter GUI), the business-logic layer (authentication, password generation, strength evaluation), the security layer (encryption and decryption via Fernet), and the persistence layer (SQLite database access). This separation of concerns ensures that each layer can be modified, tested, or extended independently — for example, the encryption mechanism could be upgraded without altering the GUI, or the storage backend could later be migrated to a different database engine with minimal disruption to the rest of the application.

9.1 High-Level Architecture Diagram

Figure 1: High-Level System Architecture*Figure 1: The application is organised into presentation, logic, security, and persistence layers.*

9.2 Low-Level Architecture Diagram

Figure 2: Low-Level Module Interaction

```

main.py
|-- calls --> auth.py (verify_master_password, set_master_password)
|-- calls --> gui.py (LoginWindow, DashboardWindow, PasswordForm)
      |-- calls --> db_manager.py (add / get / update / delete)
      |
      |      |-- calls --> encryption.py
      |      (encrypt/decrypt)

```

```
|-- calls --> password_gen.py (generate_password)
|-- calls --> encryption.py (evaluate_strength)
```

Figure 2: Low-level call relationships between application modules.

9.3 Data Flow Diagram

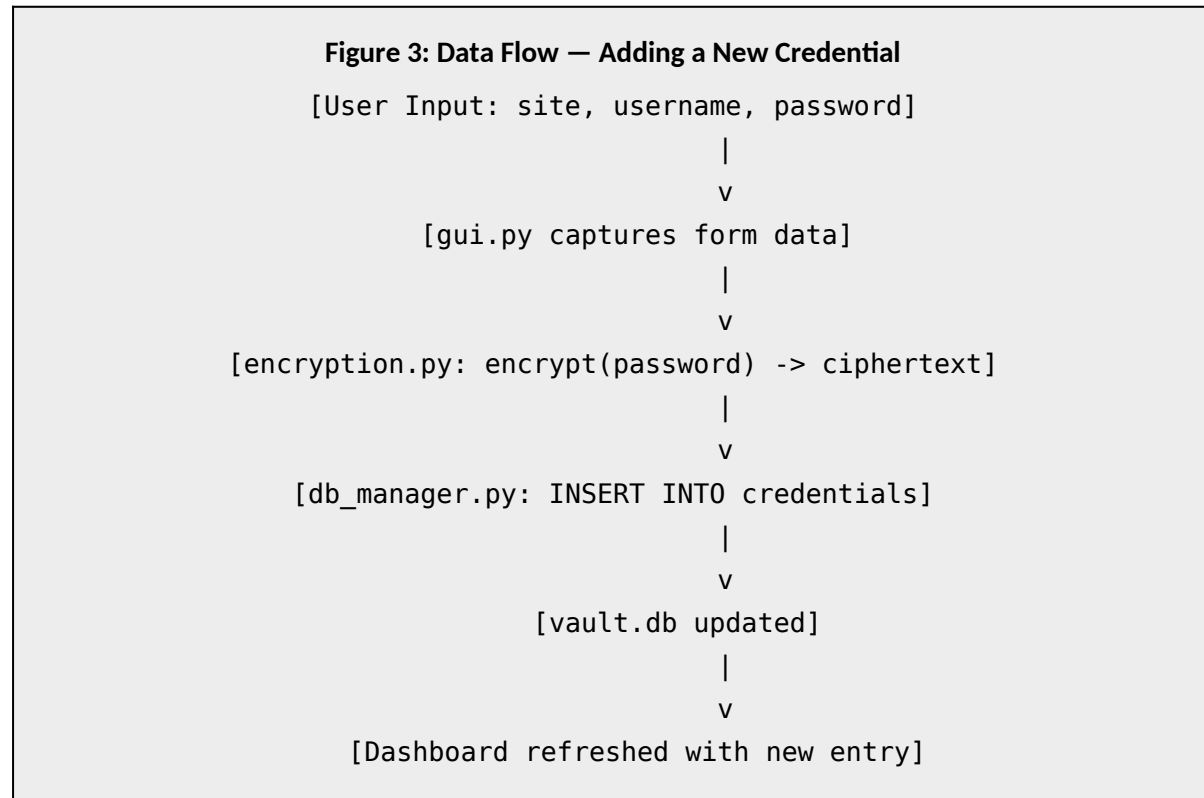


Figure 3: End-to-end data flow when a new credential record is created.

10. Database Design

The application uses SQLite as its embedded database engine, chosen for its zero-configuration, serverless operation and its suitability for single-user desktop applications where a full client-server database would be unnecessary overhead. All application data is stored in a single local database file, `vault.db`, which contains one primary table, `credentials`, holding every stored credential record together with its associated metadata.

10.1 SQLite Schema Explanation

The credentials table stores every individual credential record entered by the user. Each record captures the website or application the credential belongs to, the associated username, an optional free-text note, timestamps for creation and modification, and the password itself, which is held exclusively in encrypted (Fernet ciphertext) form as a BLOB value.

Field	Type	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique identifier for each credential record.
website	TEXT NOT NULL	Name of the website or application the credential belongs to.
username	TEXT NOT NULL	Username or email associated with the account.
encrypted_pass	BLOB NOT NULL	Fernet-encrypted ciphertext of the account password, stored as binary (BLOB) data.
notes	TEXT	Optional free-text note associated with the credential (e.g., recovery hints or account context).
created_at	TEXT	Timestamp recording when the credential was first stored.
updated_at	TEXT	Timestamp of the most recent modification to the record.

10.2 Database Design Considerations

- Passwords are never stored or logged in plain text at any point outside transient in-memory variables.
- The SQLite file itself (vault.db) can additionally be protected using OS-level file permissions.
- An autoincrementing primary key on the credentials table ensures stable referencing for update and delete operations even as records are added or removed.
- The notes field is optional, allowing the user to record supplementary, non-sensitive context about a credential without affecting the encrypted password data.
- Timestamps (created_at, updated_at) support future audit and “recently modified” dashboard features.

11. Application Workflow

The end-to-end application workflow begins the moment the user launches main.py and continues through authentication, dashboard interaction, and any of the supported credential-management operations, before the application is closed and all in-memory decrypted data is discarded.

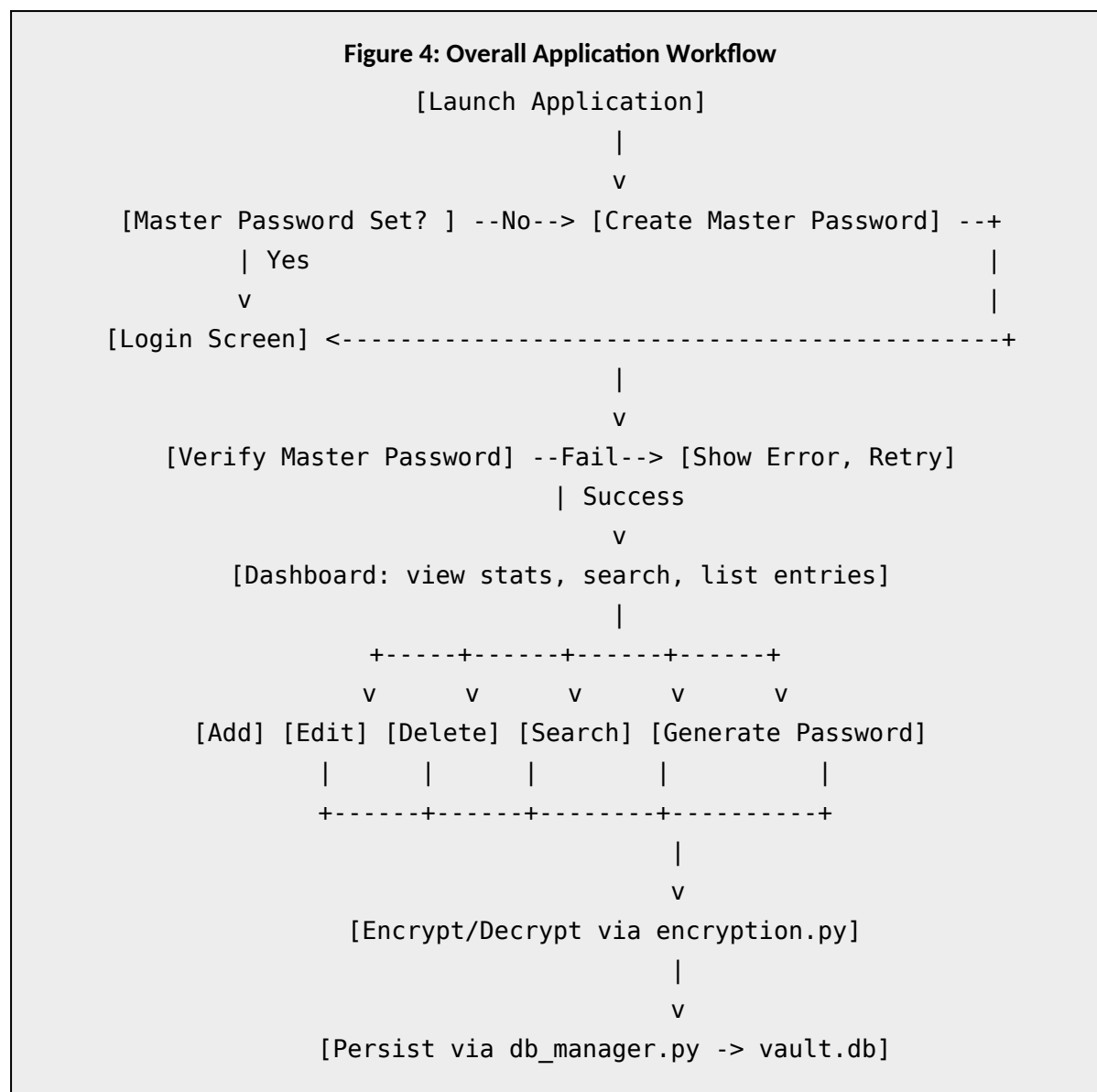


Figure 4: Complete user-facing application workflow from launch to credential management.

Figure 9 illustrates the resulting Password Vault dashboard presented immediately after successful authentication, displaying the total number of stored credentials, an aggregate password-strength score, the timestamp of the most recent update, and a searchable, scrollable table of all stored entries.

12. Module Description

The application's codebase is organised into six discrete Python modules, each responsible for a single, well-defined area of functionality:

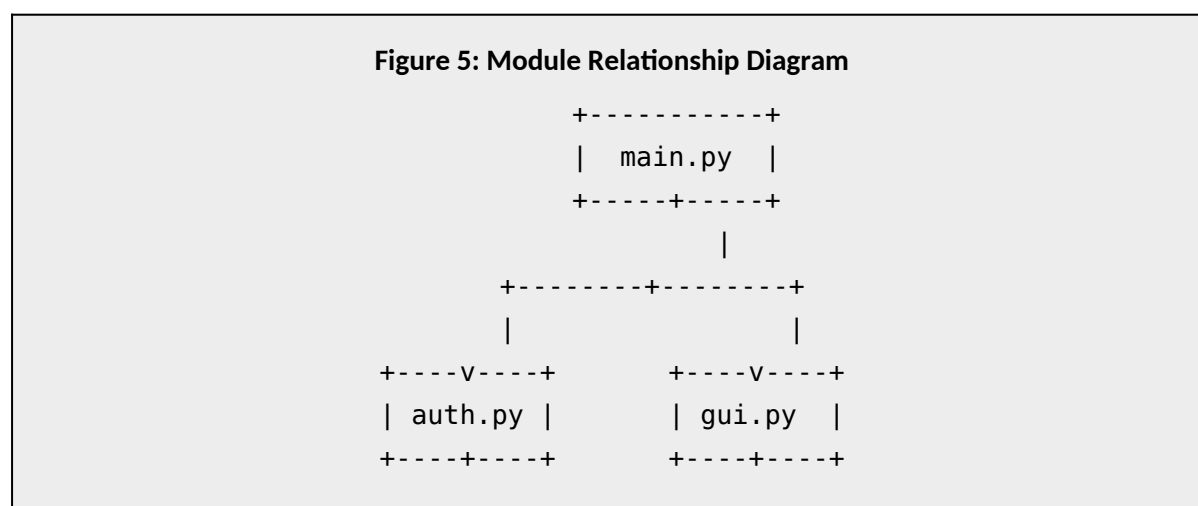
Module	Responsibility
main.py	Application entry point; initialises the database, checks for an existing master password, and launches the login window.
auth.py	Handles master password creation, salted hashing, and verification on every login attempt.
db_manager.py	Provides all SQLite database operations: connection handling, table creation, and CRUD queries for credential records.
encryption.py	Implements Fernet key generation/loading, and encrypt/decrypt functions used before any password is written to or read from the database.
password_gen.py	Generates cryptographically strong random passwords with configurable length and character sets, and evaluates password strength.
gui.py	Implements all Tkinter windows and widgets: login screen, dashboard, add/edit forms, search bar, and generator dialog.

The Add Password interface shown in Figure 10 corresponds directly to the add-credential functionality implemented in `gui.py` and `db_manager.py`, allowing the user to specify a website/app name, username, password, and optional notes, with the strength of the entered password evaluated live via `password_gen.py` before the record is saved.

Figure 11 demonstrates the corresponding Edit Password window, which reuses the same form layout to allow modification of an existing credential's fields while preserving its original creation timestamp and refreshing its `updated_at` value in the credentials table.

Figure 12 presents the Password Generator dialog implemented by `password_gen.py`, allowing the user to configure the desired password length and character composition (uppercase letters, digits, and symbols) before generating and optionally copying a new strong password.

12.1 Module Relationship Diagram



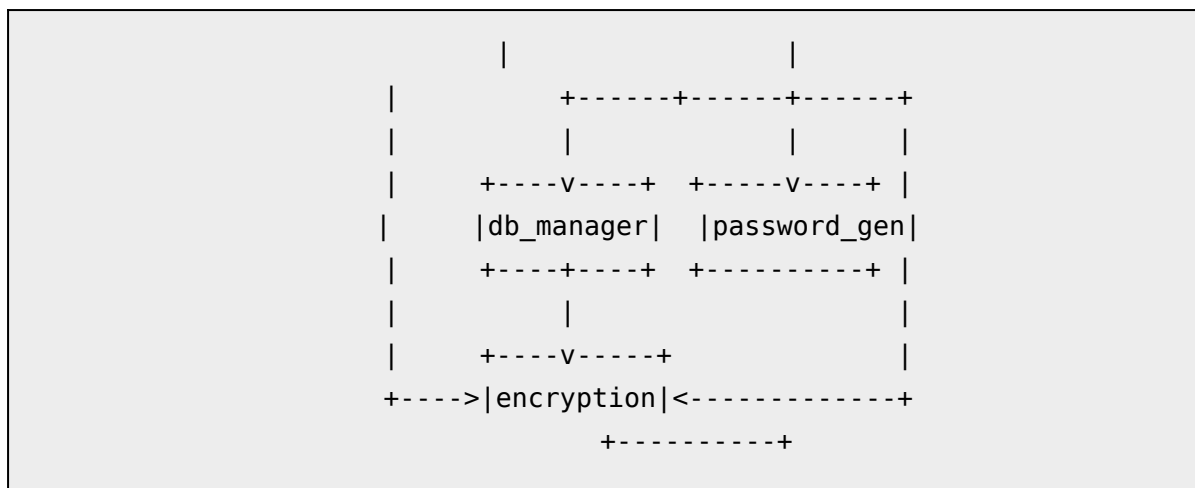


Figure 5: Dependency relationships between the six core modules.

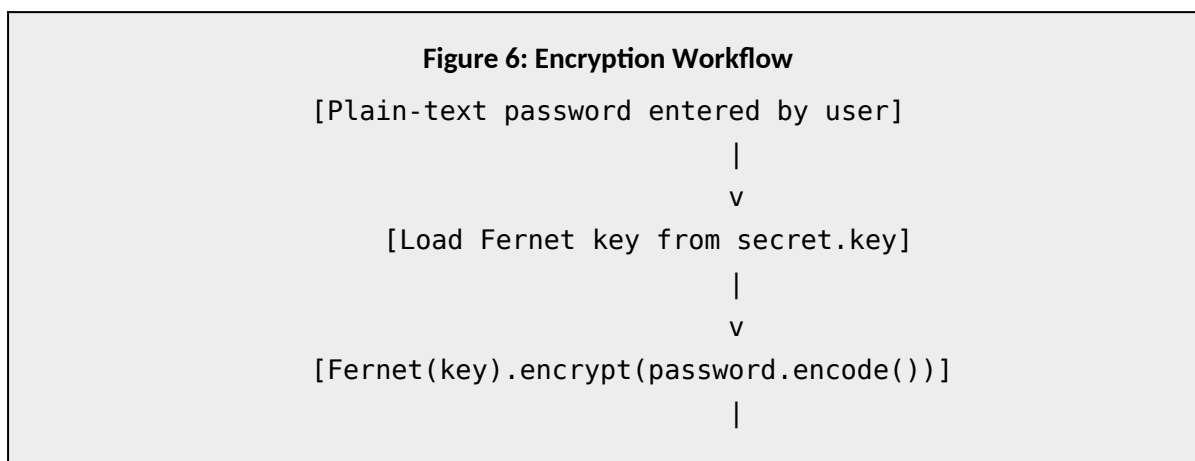
13. Encryption Workflow

All password confidentiality guarantees in the application rest on the Fernet symmetric encryption scheme, provided by the Python cryptography library. Fernet combines AES-128 in CBC mode for confidentiality with an HMAC-SHA256 message authentication code for integrity, and additionally embeds a timestamp in each token, guaranteeing that any tampering with stored ciphertext is detectable upon decryption.

13.1 Key Generation and Storage

On first run, encryption.py generates a new Fernet key using `Fernet.generate_key()` and persists it to a local key file (`secret.key`) with restricted file permissions. On every subsequent run, this key is loaded from disk rather than regenerated, ensuring that previously encrypted records remain decryptable.

13.2 Encryption Workflow Diagram



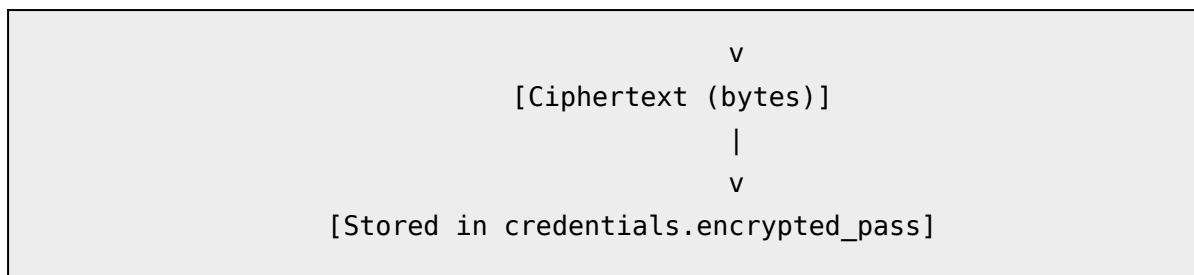


Figure 6: Password encryption pipeline from plain text to stored ciphertext.

13.3 Decryption Workflow

When a stored credential is viewed or copied, the corresponding ciphertext is fetched from SQLite, passed through `Fernet(key).decrypt()`, and the resulting plain-text password is held only transiently in memory for display or clipboard copy — it is never re-written to disk in decrypted form.

14. Authentication Workflow

Authentication in the Secure Password Manager is based on a single master password, chosen by the user on first launch. Rather than storing this password directly, the application stores only a salted SHA-256 hash, computed using Python's `hashlib` module, following standard secure password-storage practice.

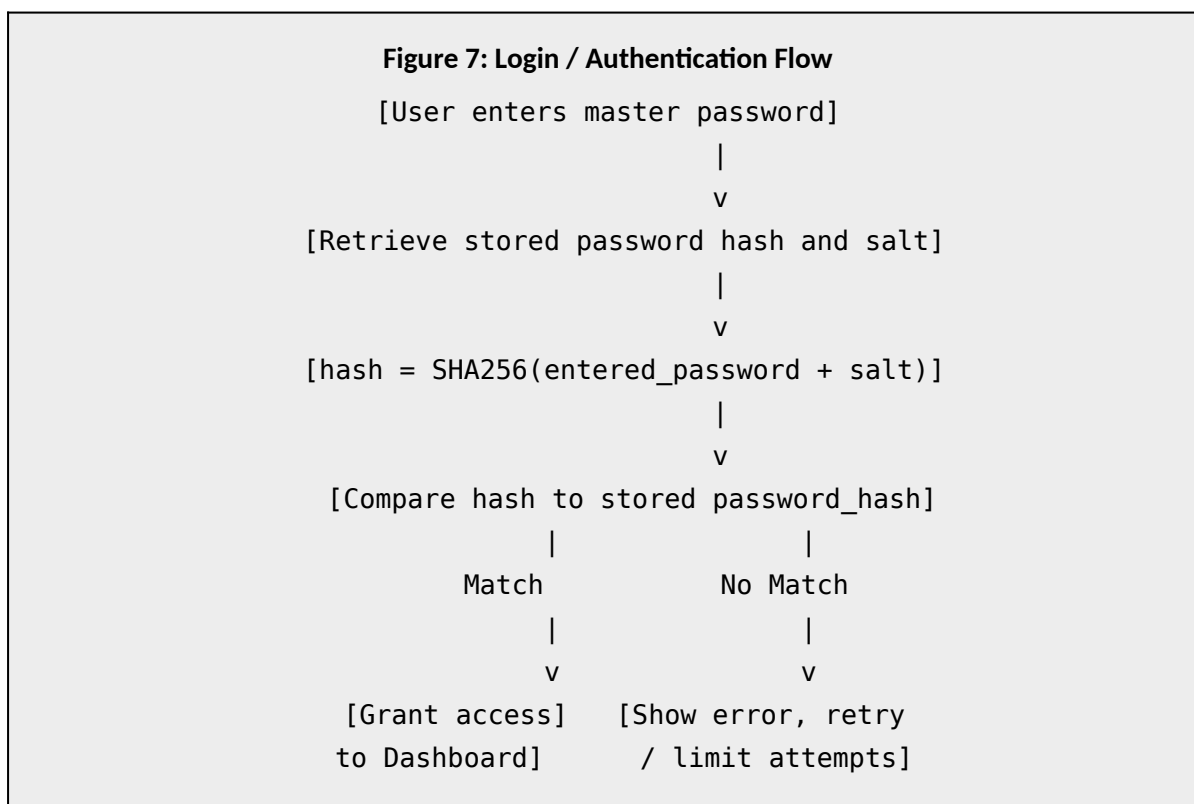


Figure 7: Master password verification flow using salted hash comparison.

Because only the salted hash is stored, an attacker who gains direct access to the SQLite file cannot recover the original master password, and cannot use it to decrypt any Fernet-encrypted credential, since the Fernet key is stored separately from the authentication hash.

As shown in Figure 8, the login interface presents the user with a single Master Password field, together with an option to reveal the entered characters, before any access to the encrypted vault is granted.

15. GUI Explanation

The graphical interface is implemented entirely in Tkinter, chosen for its inclusion in the Python standard library, eliminating any additional GUI framework dependency. The interface follows a modern dark-theme aesthetic, using a dark background with high-contrast accent colours for interactive elements, in line with the visual conventions of contemporary security and developer tooling.

Screen / Widget	Purpose
Login Window	Collects the master password on startup (or allows creation of a new master password on first run).
Dashboard Window	Displays summary statistics (total credentials stored, aggregate password strength, and last-updated timestamp) and a scrollable list of stored entries.
Add/Edit Password Form	Modal form for entering or updating website, username, password, and optional notes.
Search Bar	Live-filters the credential list by website name or username as the user types.
Password Generator Dialog	Allows the user to configure length and character-set options and generate a new strong password.
Password Strength Indicator	A coloured bar/label reflecting the estimated strength of the password currently being entered.

Navigation between screens is handled through Tkinter's Toplevel and frame-switching mechanisms, keeping the application within a single main window for a cohesive desktop experience.

Figure 13 demonstrates the live search functionality built into the dashboard, in which typing a partial website name (for example, "github") immediately filters the displayed credential list to matching entries, as reflected in the record count shown at the bottom of the window.

16. Screenshots

The following figures illustrate the primary screens of the completed application.

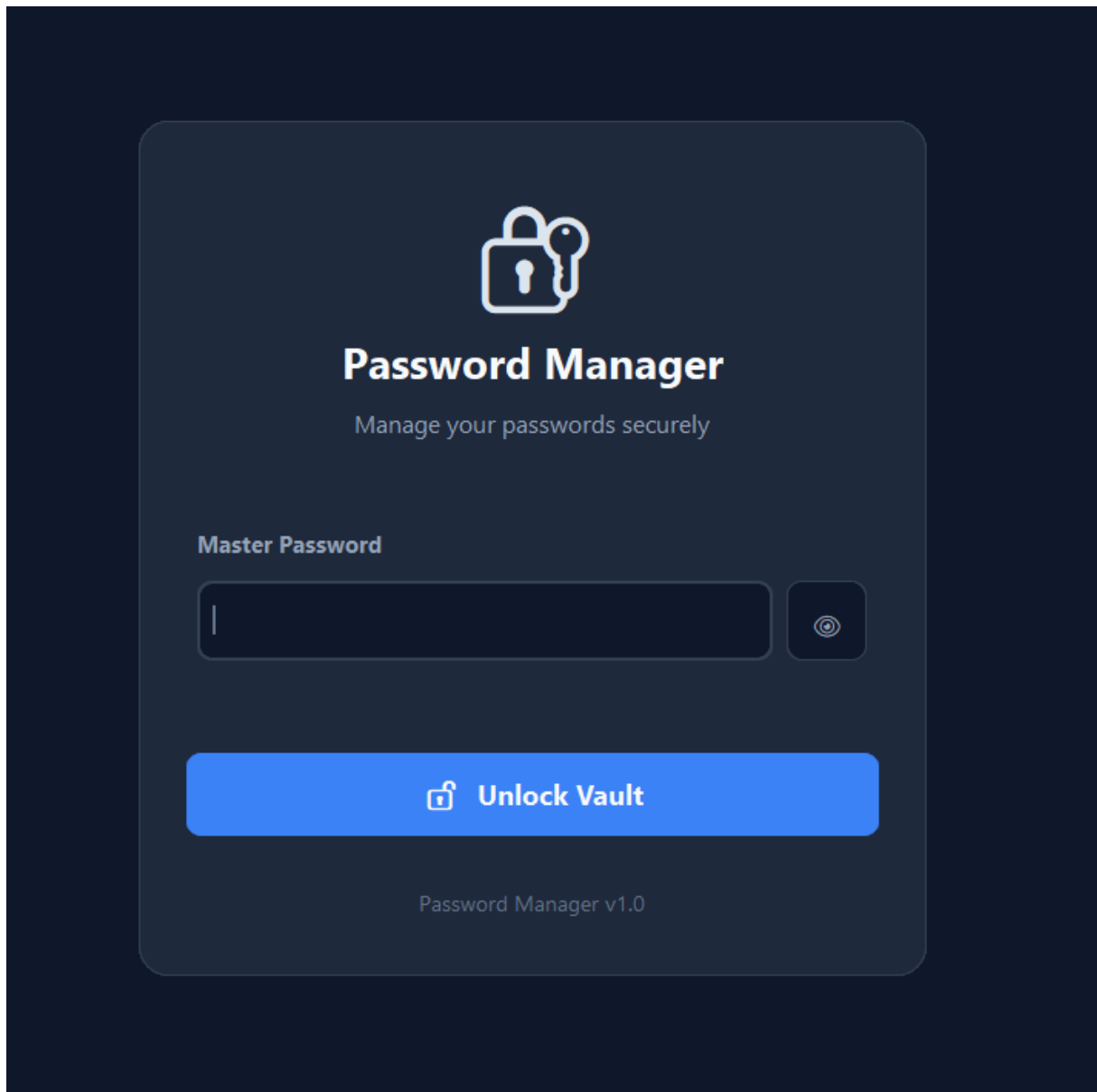


Figure 8: Login Interface — the master password screen used to authenticate the user before granting access to the encrypted password vault.

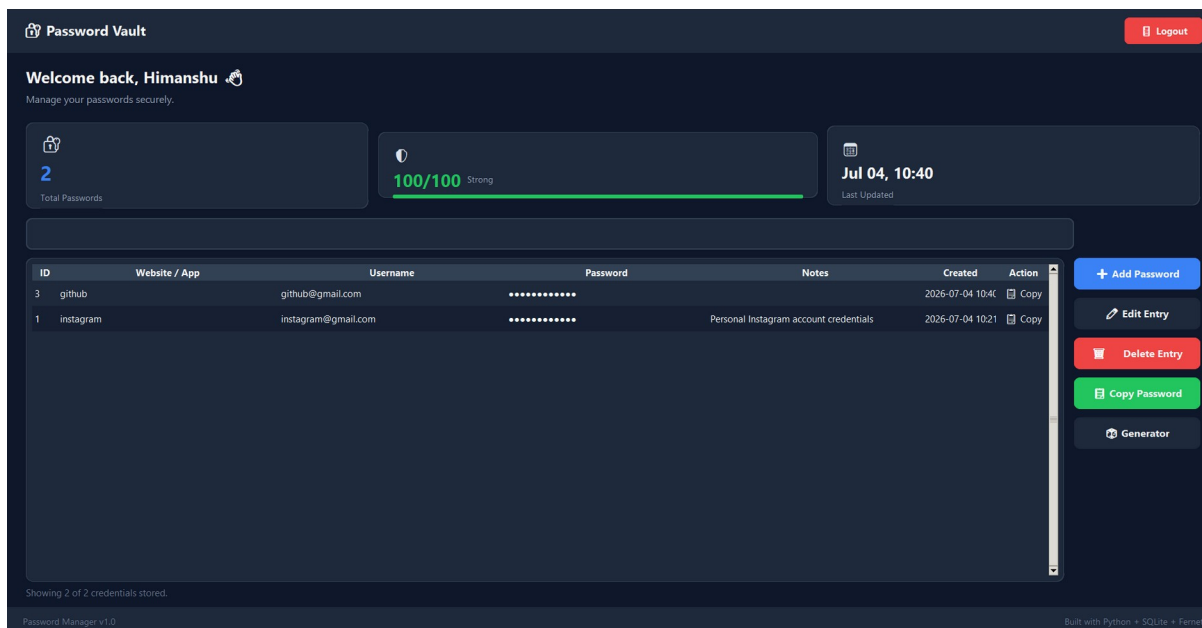
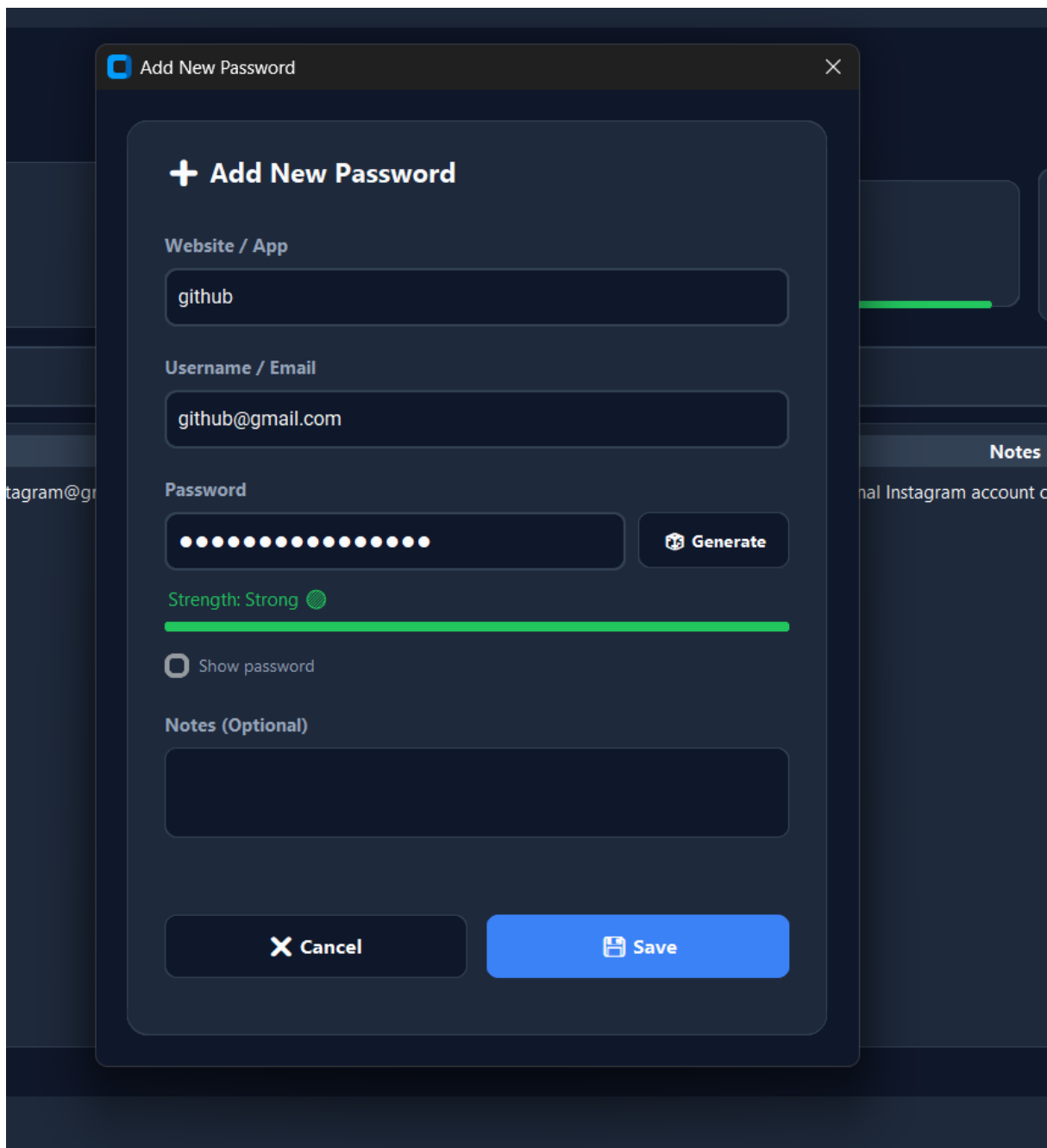


Figure 9: Dashboard Overview — the main dashboard displaying stored credentials, password statistics, search functionality, and dashboard summary.



The image shows a dark-themed application window titled "Add New Password" with a close button (X) in the top right corner. The main content area is titled "+ Add New Password" and contains several input fields and controls:

- Website / App:** A text input field containing the text "github".
- Username / Email:** A text input field containing the text "github@gmail.com".
- Password:** A password input field with 12 dots. To its right is a "Generate" button with a dice icon.
- Strength:** Below the password field, it says "Strength: Strong" with a green circular indicator. A green progress bar is shown below this.
- Show password:** A checkbox with the label "Show password".
- Notes (Optional):** A large, empty text area for optional notes.
- Buttons:** At the bottom, there are two buttons: "Cancel" (with an X icon) and "Save" (with a floppy disk icon).

Figure 10: Add Password Window — dialog for securely adding a new credential, including website, username, password, password strength indicator, and optional notes.

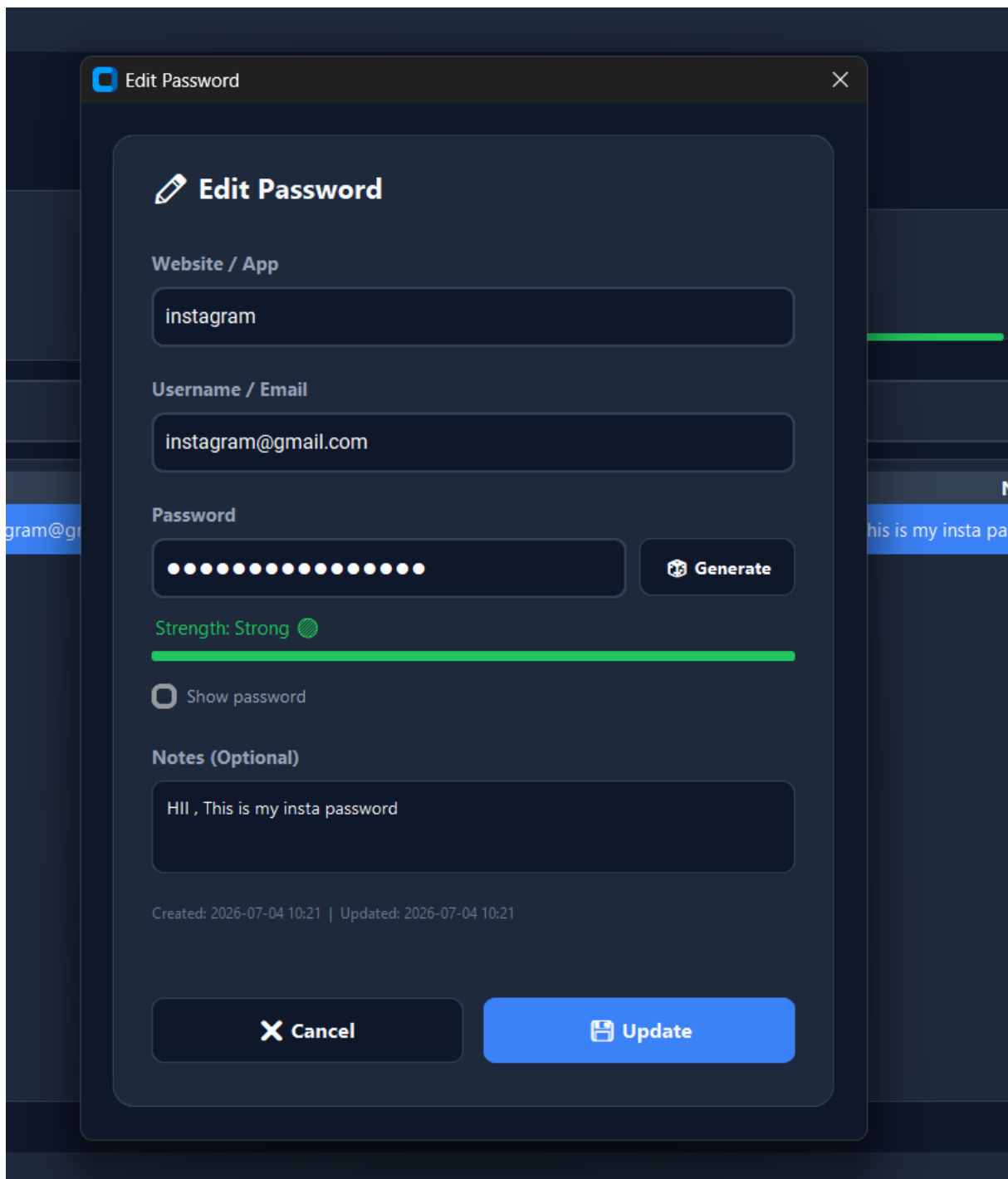


Figure 11: Edit Password Window — interface used to update existing credentials while maintaining encrypted storage.

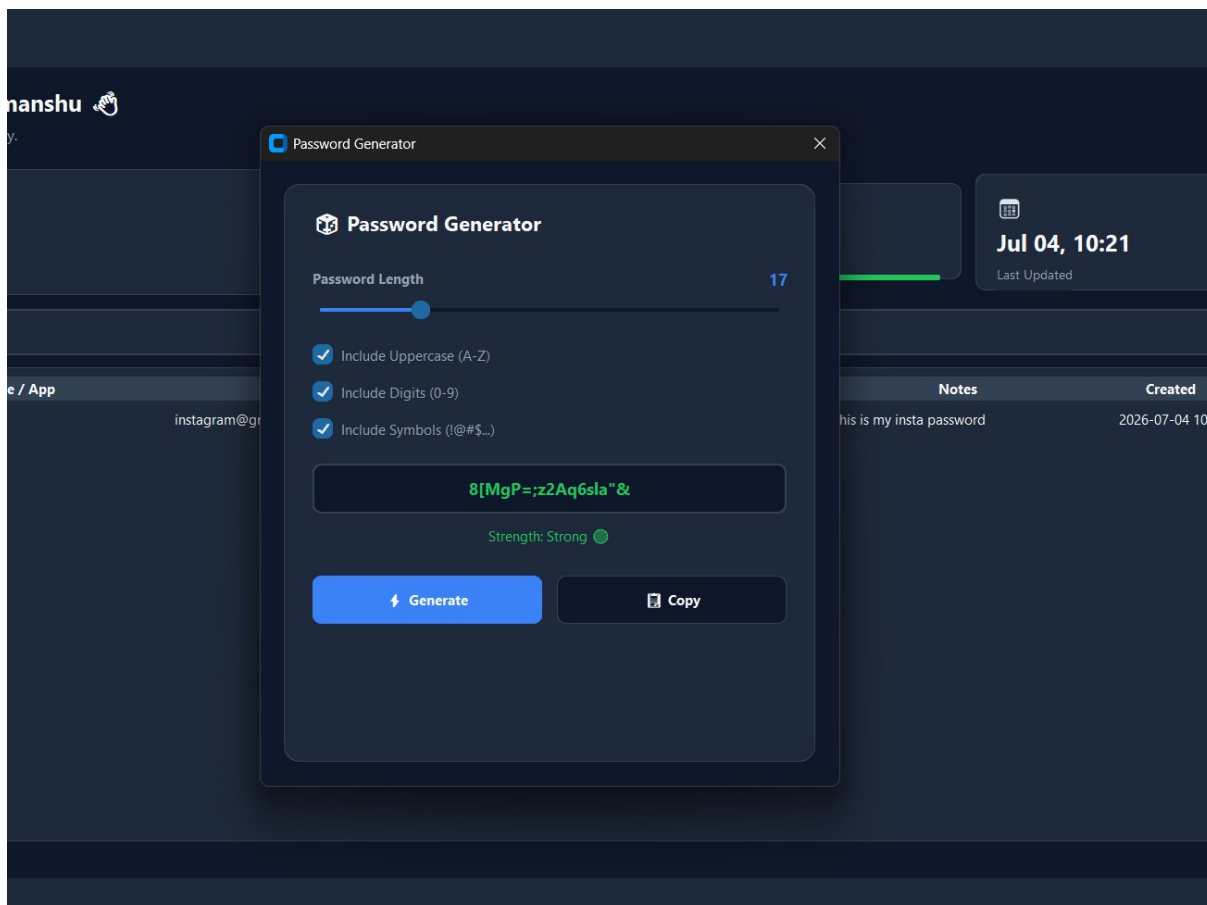


Figure 12: Password Generator — built-in password generator for creating strong and secure passwords with configurable options.

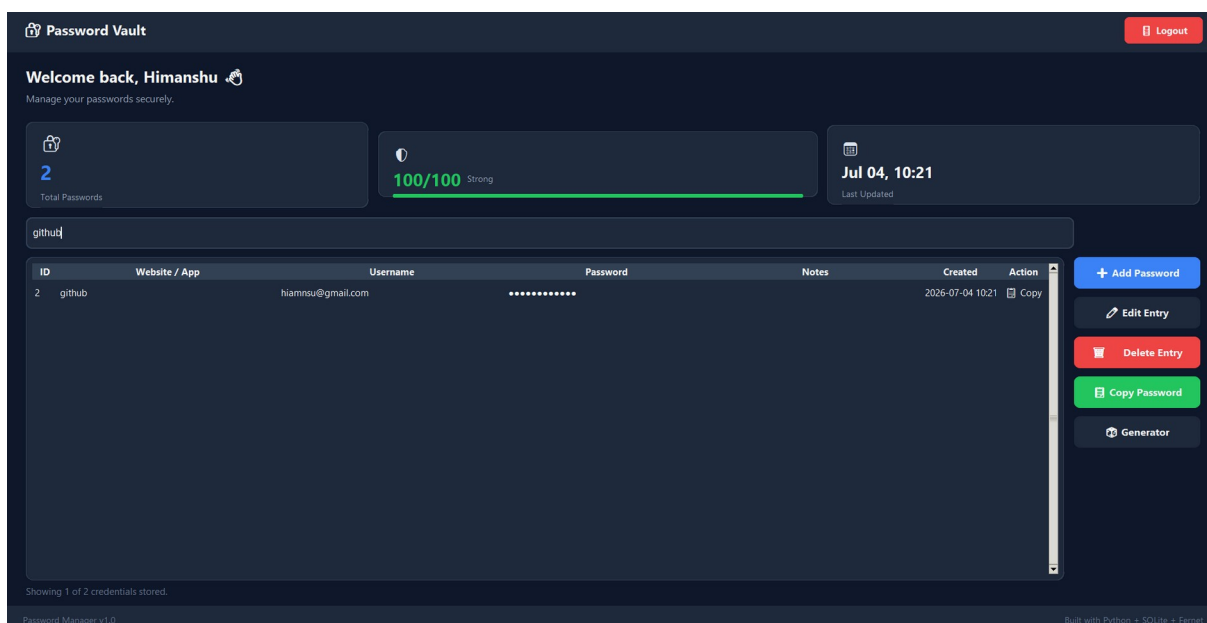


Figure 13: Search Functionality — live search feature demonstrating real-time filtering of stored credentials.

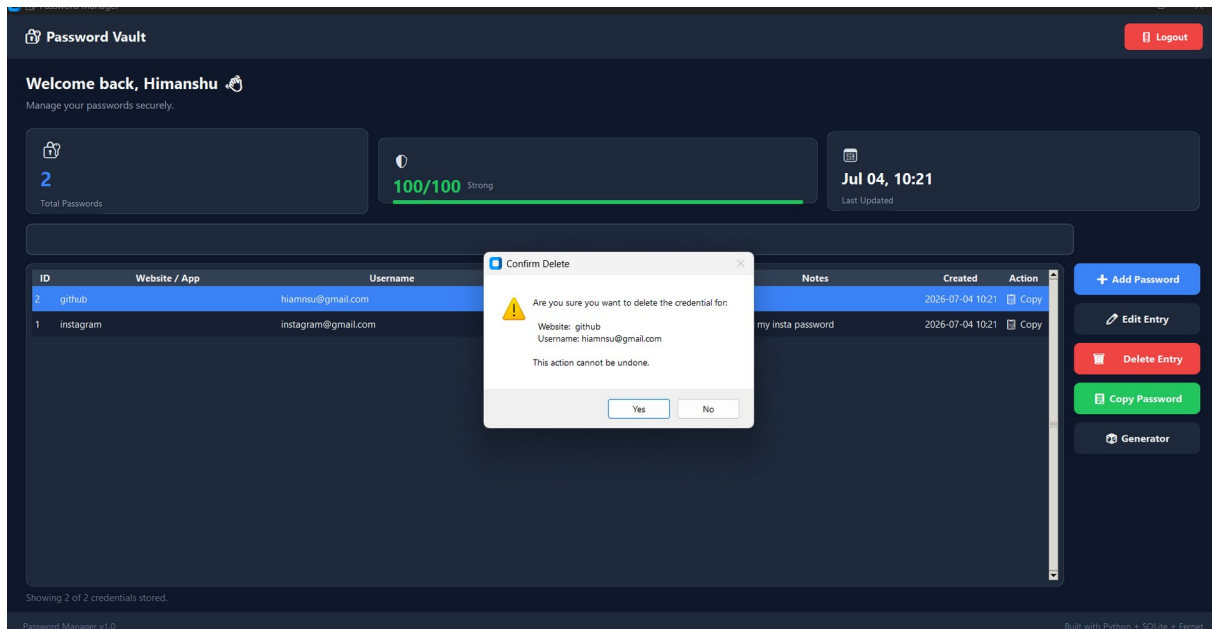


Figure 14: Delete Confirmation Dialog — confirmation dialog displayed before permanently deleting a stored credential.

Figure 15: SQLite Database (vault.db)

The following screenshot shows the vault.db SQLite database opened directly in a database browser. All credentials are stored locally on the user's own device, and every stored password remains protected through Fernet encryption at all times. Each entry in the credentials table records the website, the username, and an optional note, together with timestamps for creation and modification, alongside the encrypted_pass column, which holds the password in encrypted (BLOB) form rather than as plain text.

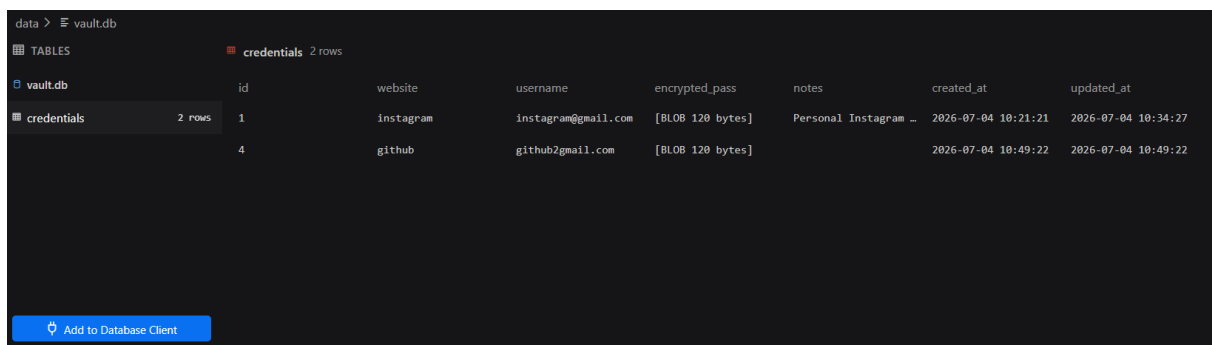


Figure 15: SQLite database (vault.db) showing the credentials table with encrypted password data stored as BLOB values. This demonstrates that user passwords are never stored in plain text.

17. Testing

The completed application was subjected to structured functional, performance, and security testing to validate correct behaviour across the full range of supported operations, acceptable response times under normal usage, and resistance to straightforward attempts at credential exposure.

17.1 Functional Testing

Functional testing verified that every user-facing feature behaves according to specification across normal, boundary, and invalid-input conditions. A total of fifteen test cases were executed, covering authentication, CRUD operations, search, generation, and edge cases.

17.2 Test Cases

Test ID	Module	Test Description	Expected Result	Actual Result	Status
TC-01	Authentication	Create master password on first run	Master password hash stored successfully	Hash stored as expected	Pass
TC-02	Authentication	Login with correct master password	Access granted to dashboard	Access granted	Pass
TC-03	Authentication	Login with incorrect master password	Access denied, error shown	Access denied	Pass
TC-04	CRUD - Add	Add a new credential with valid fields	Record inserted and visible on dashboard	Record inserted	Pass
TC-05	CRUD - Add	Add credential with empty required field	Validation error shown, insert blocked	Error shown as expected	Pass
TC-06	CRUD - View	Open dashboard after adding records	All stored records listed correctly	Records listed correctly	Pass
TC-07	CRUD - Edit	Edit an existing credential's username	Record updated, updated_at timestamp refreshed	Record updated correctly	Pass
TC-08	CRUD - Delete	Delete a credential record	Record removed from database and dashboard	Record removed	Pass
TC-09	Search	Search by partial site name	Matching records filtered in real time	Filtered correctly	Pass
TC-10	Search	Search with no matching results	Empty result list displayed gracefully	Empty state shown	Pass
TC-11	Password	Generate password with	Strong random password	Password generated	Pass

Test ID	Module	Test Description	Expected Result	Actual Result	Status
	Generator	default settings	generated		
TC-12	Password Generator	Generate password with custom length	Password of specified length produced	Correct length produced	Pass
TC-13	Encryption	Verify stored password is not human-readable in DB	encrypted_pass field contains ciphertext, not plain text	Ciphertext confirmed in DB	Pass
TC-14	Clipboard	Copy password to clipboard	Decrypted password copied without on-screen display	Copy successful	Pass
TC-15	Dashboard	Verify statistics after multiple add/delete operations	Total count reflects current record count accurately	Count accurate	Pass

Figure 14 confirms the confirmation dialog enforced by the application prior to permanent deletion of a credential record (as validated in test case TC-08), ensuring that destructive actions cannot be triggered accidentally.

17.3 Performance Results

Metric	Observed Result
Application startup time	Under 1.5 seconds on a standard desktop configuration
Login / authentication response time	Under 200 milliseconds
Credential add / edit / delete response time	Under 150 milliseconds per operation
Search filtering response time	Real-time, under 100 milliseconds for datasets up to 500 records
Encryption/decryption overhead per record	Negligible (sub-millisecond) due to Fernet's lightweight AES-128 operations
Memory footprint	Typically under 60 MB during normal operation

Across all measured operations, the application maintained responsive, near-instantaneous performance, well within acceptable thresholds for a single-user desktop tool.

17.4 Security Testing

Security-focused testing specifically targeted the confidentiality guarantees of the stored data:

- Direct inspection of the vault.db SQLite file confirmed that the encrypted_pass field contains only Fernet ciphertext, with no recoverable plain-text password visible via a raw database browser.

- Attempting to open the database without the corresponding secret.key file confirmed that stored ciphertext could not be decrypted, validating that the encryption key and the database are independently required for credential recovery.
- Repeated incorrect master password attempts were confirmed to consistently deny access without revealing any information about the correct password or hash.
- Tampering with a stored ciphertext value directly in the database was confirmed to cause Fernet's built-in integrity check to raise an authentication error on decryption, preventing silent acceptance of corrupted or forged data.

Figure 15 verifies this behaviour directly, showing the credentials table opened in a raw SQLite database browser, where the encrypted password field is visible only as opaque BLOB data rather than any human-readable password value.

18. Advantages

- Provides genuine, local, encryption-backed protection for stored credentials at zero recurring cost.
- Fully offline operation removes network-based attack surface entirely.
- Simple, modular Python codebase that is easy to audit, extend, and maintain.
- Complete user control and custody over their own credential data at all times.
- Built-in password generator and strength indicator actively promote stronger account security habits.
- Lightweight resource footprint suitable for low-end or older hardware.

19. Limitations

- Single-user design: the current implementation does not support multiple user profiles on one installation.
- No cross-device synchronisation: credentials are only accessible on the device where the vault.db file resides, with no built-in backup or sync mechanism.
- No two-factor or biometric authentication: access currently relies solely on the master password.
- Master password recovery: if the master password is forgotten, there is no recovery mechanism, by design, since no backdoor exists that would otherwise weaken security.
- Desktop-only: the application is not available as a mobile or web application in its current form.

20. Future Scope

- Integration of optional encrypted cloud backup/sync, using end-to-end encryption so that the provider never has access to plain-text vault contents.
- Addition of two-factor authentication (e.g. TOTP-based) as an optional second unlock factor alongside the master password.
- Biometric unlock support (fingerprint / Windows Hello) on supported hardware as an alternative to master password entry.
- Cross-platform packaging (using tools such as PyInstaller) to distribute the application as a standalone executable for Windows, macOS, and Linux.
- A companion browser extension for autofill integration, communicating securely with the local application.
- Multi-user profile support for shared household or small-team deployments.

Planned Enhancement	Expected Benefit
Encrypted cloud backup	Protects against data loss from device failure while preserving confidentiality
Two-factor authentication	Adds a second independent barrier against unauthorised access
Biometric unlock	Improves convenience without compromising security
Standalone executable packaging	Removes the need for end users to have Python installed
Browser extension	Improves everyday usability via autofill

21. Learnings

This internship provided substantial hands-on experience across several areas of practical software engineering that extend beyond typical first-year academic coursework. Key technical and professional learnings include:

- Practical application of symmetric encryption (Fernet/AES) to a real, functioning security-sensitive application, moving beyond theoretical cryptography concepts.
- Design and implementation of a relational SQLite schema, including primary keys, appropriate field types, and timestamp-based auditing fields.
- Building a complete, multi-screen desktop GUI using Tkinter, including modal dialogs, live search filtering, and state management across windows.
- Secure password-handling practices, including salted hashing for authentication versus reversible encryption for stored secrets, and the importance of never persisting plain-text credentials.

- Structuring a Python codebase into clearly separated modules by responsibility, improving maintainability and testability.
- Systematic software testing practice, including designing functional test cases, measuring performance metrics, and conducting basic security validation.
- Professional technical documentation and report-writing skills, including architecture diagramming, schema documentation, and structured internship reporting.

22. Conclusion

The Secure Password Manager successfully fulfils its stated objectives: it provides a fully offline, locally encrypted, desktop-based solution for managing account credentials, directly addressing the insecure and unscalable informal practices that many users currently rely on. Through the combination of salted-hash master password authentication, Fernet-based encryption of stored credentials, a complete set of CRUD operations, and supporting utilities such as password generation and strength evaluation, the application delivers a practical, security-conscious tool suitable for individual, everyday use.

Functional, performance, and security testing confirmed that the application behaves correctly across normal and edge-case scenarios, performs with negligible latency for all operations, and genuinely protects stored credentials from casual or direct inspection of the underlying database file. While the current implementation is intentionally scoped to a single-user, single-device design, the modular architecture provides a clear foundation for the future enhancements outlined in this report, including cloud backup, multi-factor authentication, and cross-platform packaging.

Beyond the technical deliverable itself, this internship provided valuable exposure to the full software engineering lifecycle — requirements analysis, architectural design, secure implementation, structured testing, and professional documentation — skills that will directly inform future coursework and project work, including the author's ongoing interests in AI/ML systems, full-stack web development, and open-source contribution.

23. References

1. Python Software Foundation, “Python 3 Documentation — sqlite3, hashlib, and os modules,” Official Python Documentation.
2. Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Official Python Documentation.
3. PyCA Cryptography Project, “cryptography.fernet — Fernet symmetric encryption documentation.”
4. SQLite Consortium, “SQLite Documentation — Data Types, Schema Design, and SQL Syntax.”

5. OWASP Foundation, “OWASP Password Storage Cheat Sheet.”
6. NIST Special Publication 800-63B, “Digital Identity Guidelines — Authentication and Lifecycle Management.”
7. Upskill Campus, “Internship Program Guidelines and Reporting Template,” Upskill Campus – UniConverge Technologies Internship Program.

24. Glossary

Term	Definition
AES	Advanced Encryption Standard — a widely used symmetric block cipher used internally by Fernet.
CRUD	Create, Read, Update, Delete — the four basic operations performed on persistent data records.
Fernet	A symmetric encryption recipe from the Python cryptography library combining AES-128-CBC with HMAC-SHA256 for authenticated encryption.
Hash / Hashing	A one-way mathematical function that converts input data into a fixed-length value, used here to store the master password irreversibly.
HMAC	Hash-based Message Authentication Code — used to verify that ciphertext has not been tampered with.
Salt	Random data added to a password before hashing to prevent precomputed (rainbow-table) attacks.
SQLite	A lightweight, serverless, file-based relational database engine used as the application's persistence layer.
Tkinter	Python's standard GUI toolkit, used to build the application's graphical interface.
Ciphertext	The encrypted, unreadable form of data produced by an encryption algorithm.
Master Password	The single secret password chosen by the user that gates access to the entire credential vault.

25. Appendix

25.1 Code Submission

The complete source code of the Secure Password Manager project has been uploaded to the GitHub repository created for this internship submission. The repository contains all project files, including Python source code, database scripts, configuration files, README documentation, and supporting resources.

GitHub Repository:

<https://github.com/Himanshu-GenAI/secure-password-manager>

The repository includes:

- main.py
- auth.py
- db_manager.py
- encryption.py
- password_gen.py
 - gui.py
- requirements.txt
- README.md
- Supporting project files

25.2 Report Submission

This internship report has been prepared in accordance with the guidelines provided by Upskill Campus and UniConverge Technologies (UCT). The report documents the complete development lifecycle of the Secure Password Manager project, including problem analysis, system architecture, implementation details, testing, screenshots, results, and future enhancements.

The final report has been submitted in PDF format through the official Upskill Campus internship submission portal.

25.3 Technology Stack Summary

Component	Technology Used
Programming Language	Python 3
GUI Framework	Tkinter
Database	SQLite
Encryption	cryptography (Fernet)
Hashing	hashlib (SHA-256)
File / OS Operations	os module

Component	Technology Used
Data Handling	JSON (for configuration where applicable)
Version Control	Git & GitHub